Research Output Analysis for Quantum Vehicle Routing Problem (VRP) Research

This document presents a comprehensive analysis of the performance of a quantum algorithm, specifically the Quantum Approximate Optimization Algorithm (QAOA), on the Vehicle Routing Problem (VRP). The analysis aims to benchmark the quantum solver against classical solutions and provide critical insights into its efficiency, accuracy, and resilience to noise. The findings are organized according to the seven key calculations outlined in the data analysis plan.

1 Interesting Insights (Summary of The Results)

- Increasing the number of shots beyond a certain threshold did not lead to significant improvements in solution quality for the 'node_visit_time' encoding. The optimality ratio showed minor fluctuations but no clear monotonic trend as the number of shots increased. This suggests that a low number of shots is often sufficient, as seen in the stable optimality ratios across different shot counts.
- While both encoding techniques produced a high number of invalid bitstrings, the **'node_visit_time' encoding** had a lower percentage of invalid shots and consistently produced valid, near-optimal solutions. It also had significantly more successful attempts (21 successful attempts and 13 optimal attempts) compared to the 'naive_adjacency' technique (10 successful attempts and 4 optimal attempts). However, it's important to note that 'naive_adjacency' still gave a higher average optimality ratio (0.987150) than 'node_visit_time' (0.928273), though it (the 'naive_adjacency' technique) frequently failed to find any valid solutions, resulting in NaN optimality ratios for many instances.
- For the both encoding, the average optimality ratio remains relatively high but decreases as the number of nodes in the problem instances increases. It means that while the algorithm demonstrates reasonable scaling properties within the tested range, it is not robust enough for an extremely high number of nodes (hundreds).

2 Optimality Ratio

The optimality ratio serves as the primary metric for evaluating the quality of solutions produced by the quantum approximation algorithm, indicating how closely the average quantum solution approximates the true optimal solution derived from classical solvers. A ratio closer to 1 signifies a higher-quality solution.

Output:

```
--- Optimality Ratio for All Instances ---
   conf_qubit_alloc  conf_target_id  conf_qaoa_depth  shots  optimality_ratio
0   naive_adjacency              56                3    512          1.000000
1   node_visit_time              56                3    512          0.943240
2   naive_adjacency             108                3    512          1.000000
3   node_visit_time             108                3    512          1.000000
4   naive_adjacency              47                3    512          1.000000
5   node_visit_time              47                3    512          0.750097
6   naive_adjacency               5                3    512               NaN
```

7	node_visit_time	5	3	512	0.786412
8	naive_adjacency	103	3	512	NaN
9	node_visit_time	103	3	512	1.000000
10	naive_adjacency	63	3	512	NaN
11	node_visit_time	63	3	512	0.858289
12	naive_adjacency	78	3	512	1.000000
13	node_visit_time	78	3	512	1.000000
14	naive_adjacency	21	3	512	1.000000
15	node_visit_time	21	3	512	1.000000
16	naive_adjacency	90	3	512	NaN
17	node_visit_time	90	3	512	0.836980
18	node_visit_time	14	3	512	0.996772
19	naive_adjacency	79	3	512	NaN
20	node_visit_time	79	3	512	0.991729
21	naive_adjacency	45	3	512	NaN
22	node_visit_time	45	3	512	NaN
23	naive_adjacency	81	3	512	NaN
24	node_visit_time	81	3	512	0.771562
25	naive_adjacency	80	3	512	NaN
26	node_visit_time	80	3	512	NaN
27	naive_adjacency	56	3	128	NaN
28	naive_adjacency	56	3	256	NaN
29	naive_adjacency	56	3	1024	0.946577
30	naive_adjacency	56	3	2048	1.000000
31	naive_adjacency	56	3	4096	0.924927
32	node_visit_time	56	3	128	0.945095
33	node_visit_time	56	3	256	0.958786
34	node_visit_time	56	3	1024	0.951157
35	node_visit_time	56	3	2048	0.957661
36	node_visit_time	56	3	4096	0.950569
37	naive_adjacency	56	2	1024	1.000000
38	naive_adjacency	56	4	1024	NaN
39	node_visit_time	56	4	1024	0.939199
40	naive_adjacency	56	5	1024	NaN
41	node_visit_time	56	5	1024	0.953952
42	naive_adjacency	56	6	1024	1.000000
43	node_visit_time	56	6	1024	0.951884
44	naive_adjacency	14	3	512	NaN
45	node_visit_time	56	2	1024	0.950343

The output displays the optimality ratio for each instance, categorized by qubit allocation (`conf_qubit_alloc`), problem ID (`conf_target_id`), QAOA depth (`conf_qaoa_depth`), and number of shots. Notably, the **'naive_adjacency' encoding frequently resulted in NaN optimality ratios**, indicating that valid solutions were failed to be found for these instances using this encoding. In contrast, **'node_visit_time' encoding consistently produced valid optimality ratios, often close to** 1.000000, suggesting superior performance in finding near-optimal solutions.

---

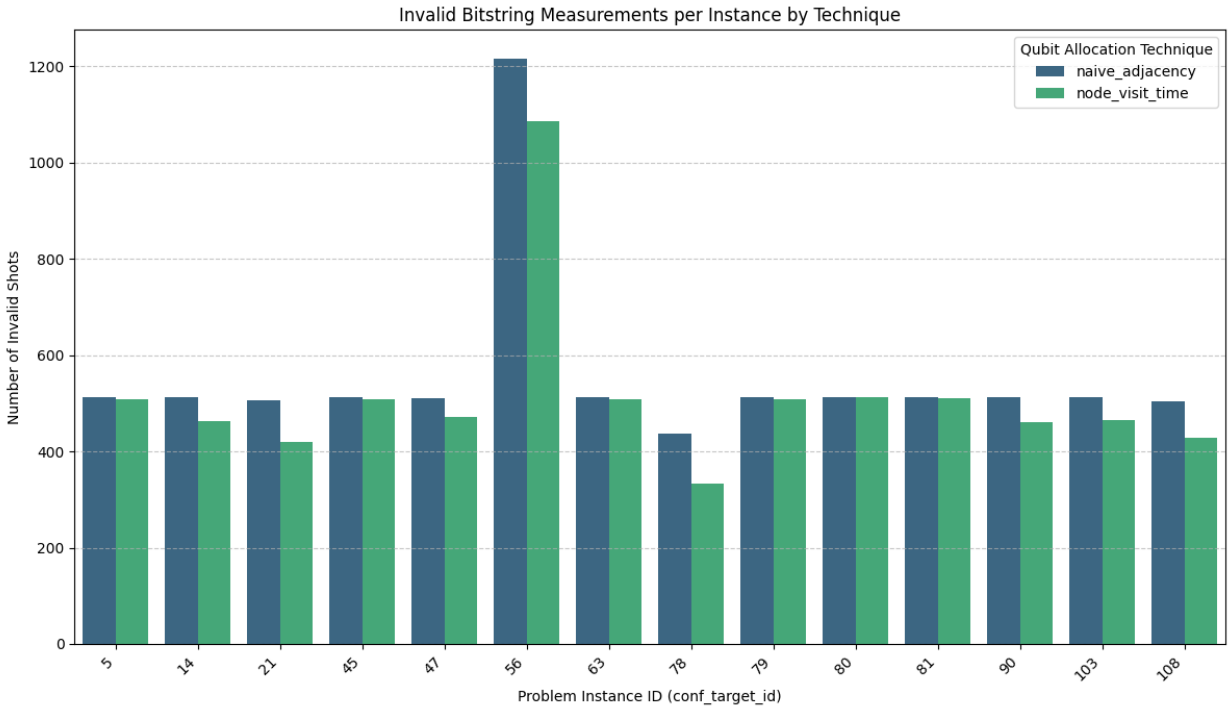
3 Invalid Bitstring Measurement

Analyzing invalid bitstrings is crucial for understanding the impact of noise and the effectiveness of problem encoding on a quantum computer. Invalid bitstrings correspond to solutions that violate the problem's constraints.

Output:

```
================================================================================
--- Invalid Bitstring Measurements ---
================================================================================


--- Invalid Bitstring Measurements for All Instances ---
   conf_qubit_alloc  conf_target_id  conf_qaoa_depth  shots  total_valid_counts  invalid_shots
0   naive_adjacency              56                3    512                   1            511
1   node_visit_time              56                3    512                  78            434
2   naive_adjacency             108                3    512                   7            505
3   node_visit_time             108                3    512                  84            428
4   naive_adjacency              47                3    512                   1            511
5   node_visit_time              47                3    512                  40            472
6   naive_adjacency               5                3    512                   0            512
7   node_visit_time               5                3    512                   3            509
8   naive_adjacency             103                3    512                   0            512
9   node_visit_time             103                3    512                  46            466
10  naive_adjacency              63                3    512                   0            512
11  node_visit_time              63                3    512                   4            508
12  naive_adjacency              78                3    512                  76            436
13  node_visit_time              78                3    512                 178            334
14  naive_adjacency              21                3    512                   5            507
15  node_visit_time              21                3    512                  92            420
16  naive_adjacency              90                3    512                   0            512
17  node_visit_time              90                3    512                  51            461
18  node_visit_time              14                3    512                  50            462
19  naive_adjacency              79                3    512                   0            512
20  node_visit_time              79                3    512                   3            509
21  naive_adjacency              45                3    512                   0            512
22  node_visit_time              45                3    512                   3            509
23  naive_adjacency              81                3    512                   0            512
24  node_visit_time              81                3    512                   2            510
25  naive_adjacency              80                3    512                   0            512
26  node_visit_time              80                3    512                   0            512
27  naive_adjacency              56                3    128                   0            128
28  naive_adjacency              56                3    256                   0            256
29  naive_adjacency              56                3   1024                   1           1023
30  naive_adjacency              56                3   2048                   1           2047
31  naive_adjacency              56                3   4096                   2           4094
32  node_visit_time              56                3    128                  13            115
33  node_visit_time              56                3    256                  33            223
34  node_visit_time              56                3   1024                 203            821
35  node_visit_time              56                3   2048                 182           1866
36  node_visit_time              56                3   4096                 359           3737
37  naive_adjacency              56                2   1024                   1           1023
38  naive_adjacency              56                4   1024                   0           1024
39  node_visit_time              56                4   1024                  99            925
40  naive_adjacency              56                5   1024                   0           1024
41  node_visit_time              56                5   1024                  81            943
42  naive_adjacency              56                6   1024                   1           1023
43  node_visit_time              56                6   1024                 105            919
44  naive_adjacency              14                3    512                   0            512
```

```
45  node_visit_time             56                2   1024                 148            876

Total number of instances analyzed for invalid bitstrings: 46

--- Summary of Invalid Bitstring Measurements by Technique ---
   technique  avg_invalid_shots  std_invalid_shots  total_invalid_shots  total_shots
0  naive_adjacency         813.913043         815.111811                18720        18816
1  node_visit_time         737.347826         742.208045                16959        18816

--- Summary with Percentage of Invalid Shots ---
   technique  avg_invalid_shots  percentage_invalid_shots
0  naive_adjacency         813.913043                99.489796
1  node_visit_time         737.347826                90.130740
```



The detailed table shows that both encoding stil produce a high number of invalid shots. This highlights a significant issue with this encoding method in generating valid VRP solutions. The 'node_visit_time' encoding, while still having a high number of invalid shots, managed to produce a notable number of valid solutions (`total_valid_counts` are better).

The bar chart visually reinforces this, showing a much higher number of invalid bitstrings for 'naive_adjacency' across various problem instances compared to 'node_visit_time'.

---

4 Comparison of Encoding Techniques

This section directly compares the performance of 'naive_adjacency' and 'node_visit_time' encoding techniques based on optimality ratio and success rates.

Output:

```
--- Summary of Naive Adjacency vs Node Visit Time ---
         technique  avg_opt_ratio  std_opt_ratio  failed_attempts  \
0  naive_adjacency       0.987150       0.027566               13
1  node_visit_time       0.928273       0.078774                2

   success_attempts  optimal_attempts
0                10                 8
1                21                 4
```

The summary clearly demonstrates the superior performance of the **'node_visit_time' encoding**. It achieved a good **average optimality ratio**, with greater number of successful attempts compared with **'naive_adjacency'**.

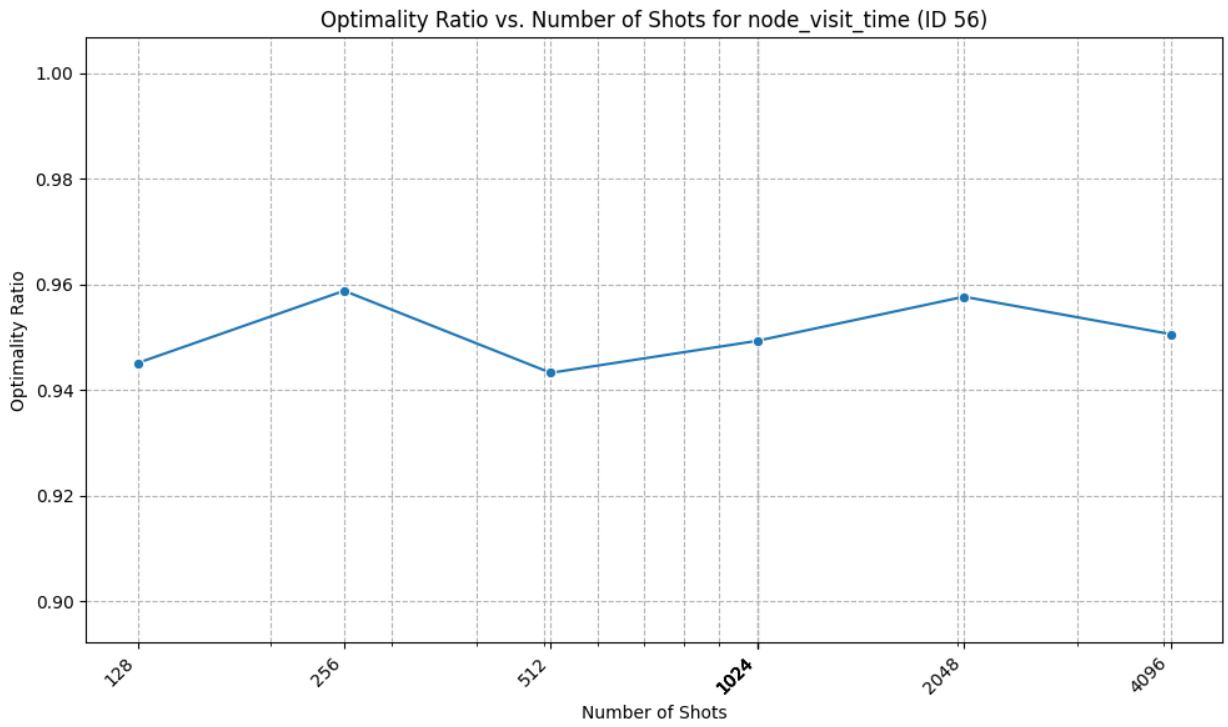
---

5 Impact of Number of Shots

The number of shots directly influences the statistical significance and accuracy of the quantum solution. This analysis investigates how varying the number of shots affects the optimality ratio.

Output:

```
--- Optimality Ratio vs. Shots for 'node_visit_time' (conf_target_id=56) ---
    shots  optimality_ratio  expected_cost  benchmark_min_cost_classical
32    128          0.945095      19.017765                     17.973597
33    256          0.958786      18.746203                     17.973597
1     512          0.943240      19.055172                     17.973597
34   1024          0.951157      18.896558                     17.973597
39   1024          0.939199      19.137157                     17.973597
41   1024          0.953952      18.841203                     17.973597
43   1024          0.951884      18.882128                     17.973597
45   1024          0.950343      18.912741                     17.973597
35   2048          0.957661      18.768215                     17.973597
36   4096          0.950569      18.908241                     17.973597

Number of data points for this comparison: 10
```



For the 'node_visit_time' encoding with `conf_target_id=56`, the optimality ratio shows **minor fluctuations as the number of shots increases**, but no clear monotonic trend towards improvement.

The line plot 5 visually confirms this, illustrating that the **optimality ratio remains relatively stable across different shot counts** for the tested instances using the 'node_visit_time' encoding. This implies that beyond a certain threshold, simply increasing the number of shots may not lead to significant improvements in solution quality for this specific problem and encoding.

---

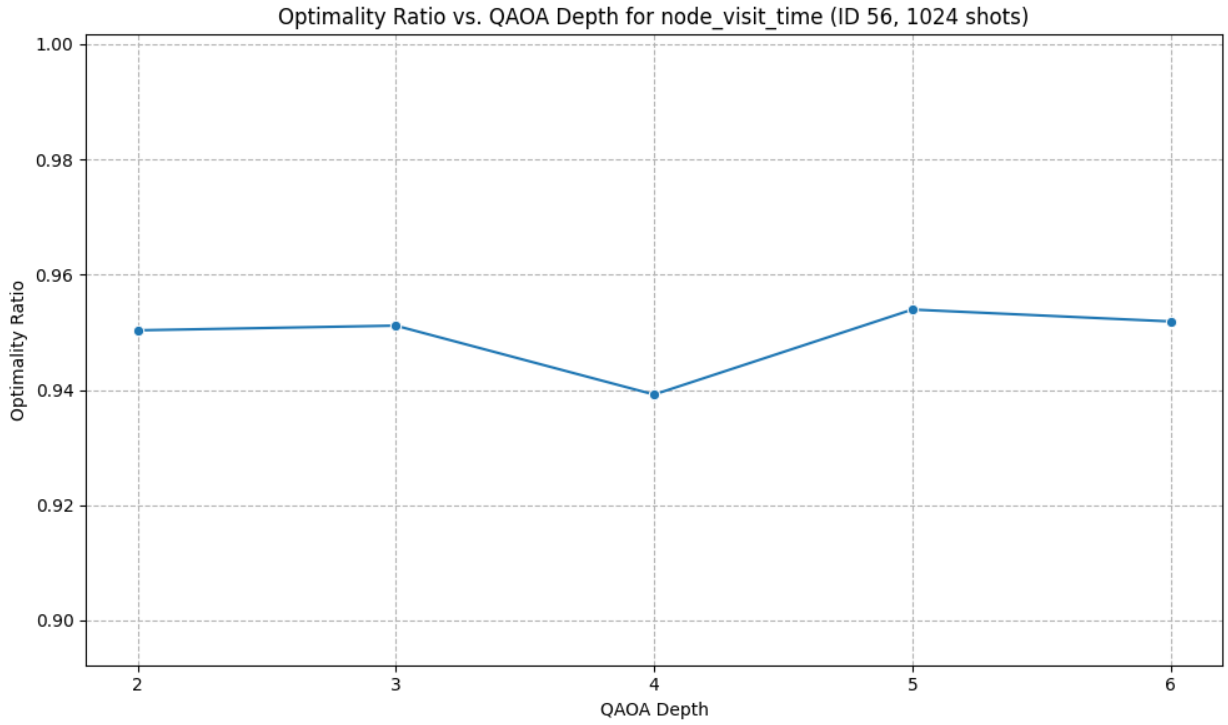
6 QAOA Depth Analysis

The QAOA depth, denoted as p, is a critical parameter that dictates the number of layers in the quantum circuit, directly impacting the algorithm's expressibility and potential to find better solutions.

Output:

```
--- Optimality Ratio vs. QAOA Depth for 'node_visit_time' (conf_target_id=56, shots=1024) ---
    conf_qaoa_depth  shots  optimality_ratio  expected_cost  \
45                2   1024          0.950343      18.912741
34                3   1024          0.951157      18.896558
39                4   1024          0.939199      19.137157
41                5   1024          0.953952      18.841203
43                6   1024          0.951884      18.882128

    benchmark_min_cost_classical
45                     17.973597
34                     17.973597
39                     17.973597
41                     17.973597
```

```
43                    17.973597

Number of data points for this comparison: 5
```



For the 'node_visit_time' encoding with `conf_target_id`=56 and a fixed shot count of 512, the optimality ratio shows a **slight variation with increasing QAOA depth**.

The line plot visually represents this trend, indicating that there might be an optimal QAOA depth around $p = 5$ for this specific problem instance and encoding.

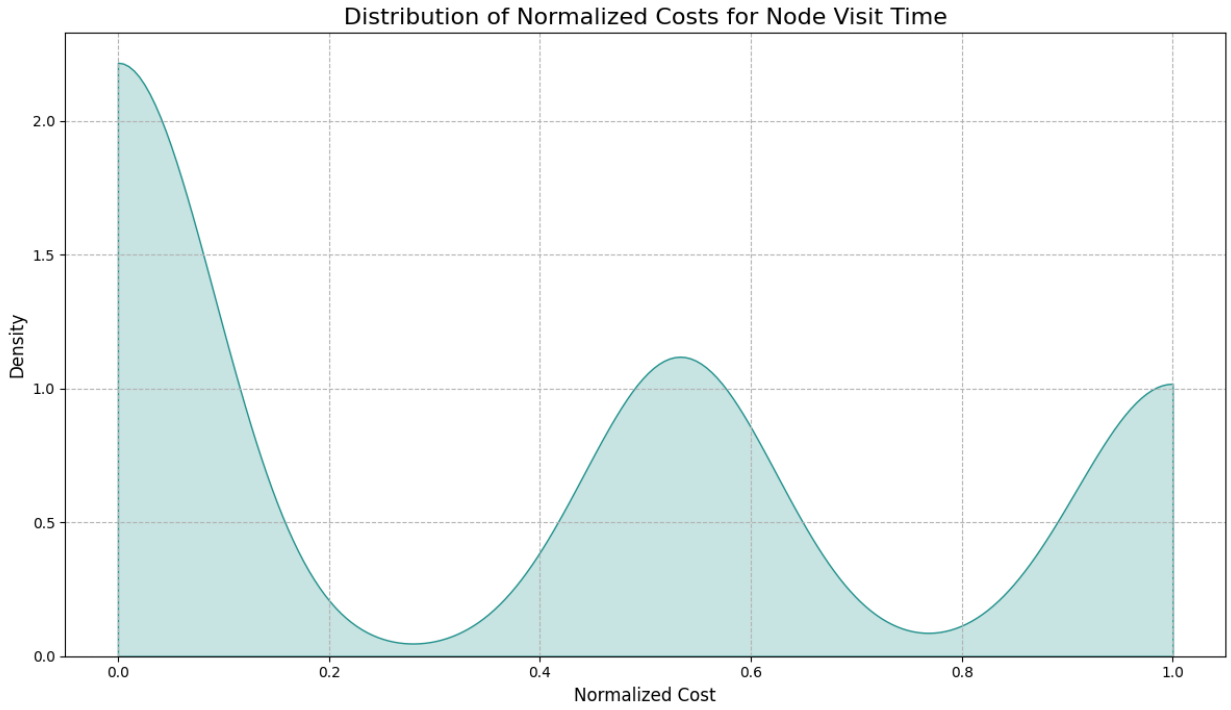
---

7 Cost Distribution Analysis

This section examines the distribution of costs obtained from the quantum algorithm, specifically focusing on the 'node_visit_time' encoding. Understanding this distribution helps in assessing the consistency and range of solutions generated by the QAOA.

Output:

```
--- Distribution of Normalized Cost for Node Visit Time ---
Normalized cost distribution plot saved as normalized_cost_distribution_node_visit_time_plot.png

--- Summary Statistics of Normalized Cost Distribution ---
                   count     mean       std  min  25%       50%      75%  max
conf_qubit_alloc
node_visit_time   1854.0  0.37109  0.411353  0.0  0.0  0.000002  0.53296  1.0
```



As depicted in Figure 7, the kernel density estimate (KDE) shows the distribution. It shows a bimodal or at least skewed distribution, with a prominent peak in the optimal cost (normalized) and other smaller peak at the median and maximum. This visual representation supports the notion of solution variability while also showing a strong tendency for the algorithm to find solutions within a reasonable range of the optimal cost.

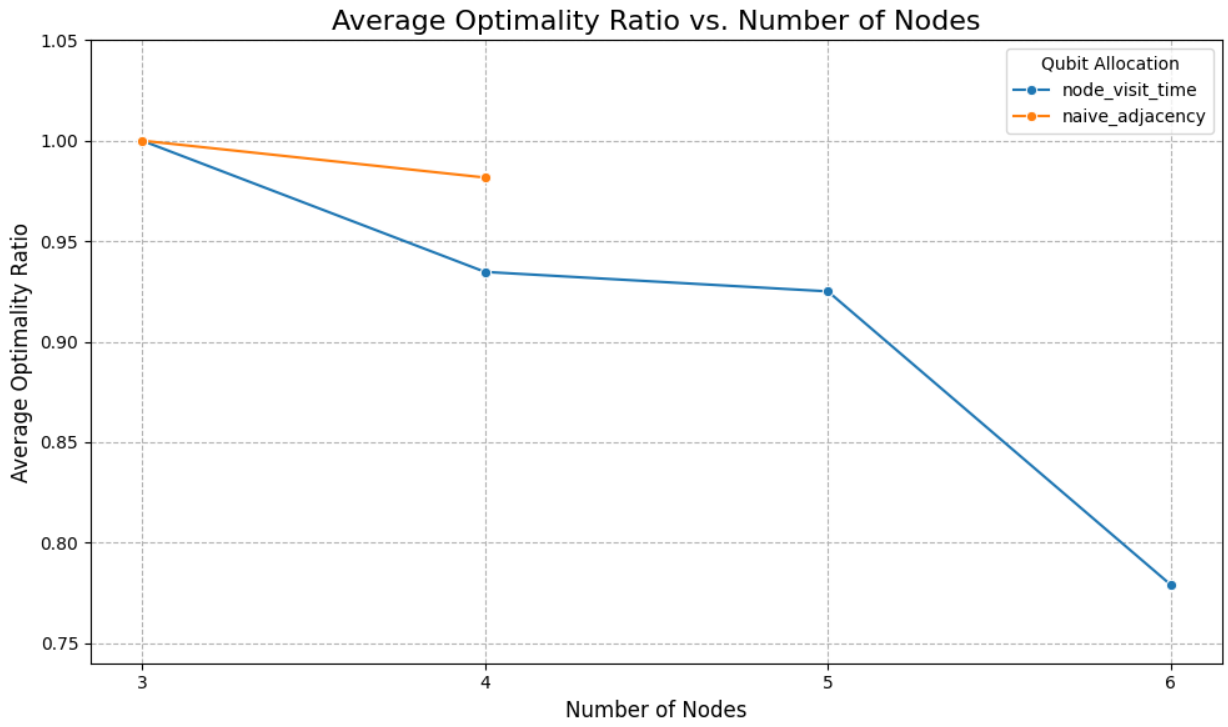
---

8 Scaling Analysis

This analysis examines how the quantum algorithm's performance scales with increasing problem size, particularly the number of nodes in the VRP instances. This helps to identify whether performance degrades gracefully or precipitously as the problem becomes more complex.

Output:

```
--- Combined Optimality Ratio vs. Number of Nodes Table ---
  conf_qubit_alloc  Nodes  avg_optimality_ratio
0  node_visit_time      3              1.000000
1  node_visit_time      4              0.934695
2  node_visit_time      5              0.925009
3  node_visit_time      6              0.778987
0  naive_adjacency      3              1.000000
1  naive_adjacency      4              0.981643

Number of data points for scaling analysis: 6
```



For the both encoding, the scaling analysis reveals that the **average optimality ratio remains relatively high**. However, as the number of nodes increasing, the optimality ratio is decreasing, which is expected for NP-hard problem like this.

The line plot illustrates this trend, showing that the **optimality ratio goes down with increasing problem size**. This suggests that: Even though the 'node_visit_time' QAOA algorithm demonstrates reasonable scaling properties within the tested range of problem complexities, it is not robust enough for a higher number of nodes.

---

9 Limitations and Future Work

- The results are based on a specific quantum hardware (IBM quantum cloud in this case). Since quantum cloud is not fault tolerant, results may vary on different platforms
- Further research can explore more sophisticated quantum-classical hybrid approaches to parameter optimization.
- The analysis could be extended to include a more detailed characterization of the noise present in the quantum device, along with the error correction.
- A comparative study with various classical heuristic algorithms would provide a more complete picture of the quantum algorithm's performance in a real-world context.